

TECH FORCE: GALACTIC
ARCHITECTURE

BUFFER OVERFLOW (BOF)



10

LISTA DE FIGURAS

Figura 1 - <i>Stack</i> layout antes e depois	11
Figura 2 - Representação do código vulnerável em Assembly.....	12
Figura 3 - Passando argumentos via debugger	12
Figura 4 - Module: overflow.exe	12
Figura 5 - Encontrando a mensagem de erro.....	13
Figura 6 - Função main encontrada no x64dbg.....	13
Figura 7 - Adicionando um breakpoint na função strcpy.....	14
Figura 8 - Executando até a função strcpy.....	14
Figura 9 - <i>Stack</i> layout antes da execução da função strcpy	14
Figura 10 - <i>Stack</i> layout com offsets relativos antes da execução a função strcpy..	15
Figura 11 - <i>Stack</i> layout com offsets relativos antes da execução a função strcpy (2)	16
Figura 12 - Retorno da função strcpy para main	16
Figura 13 - O endereço de retorno da função antes da main (entrypoint) está sobrescrito na <i>stack</i>	18
Figura 14 - EIP overflow	18
Figura 15 - Download da versão 2.7.0 do python.....	20
Figura 16 - Download da versão 32 e 64 bits.....	20
Figura 17 - Copiando os plugins.....	21
Figura 18 - Configurando o x64dbgpy	21
Figura 19 - x64dbgpy configurado corretamente.....	22
Figura 20 - Mensagem de help do plugin mona	22
Figura 21 - Vulnserver em execução.....	23

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo de código vulnerável 1	7
Código-fonte 2 – Exemplo de código vulnerável 2	8
Código-fonte 3 – Exemplo de código vulnerável 3	8
Código-fonte 4 – Exemplo de código vulnerável 4	9
Código-fonte 5 – Exemplo de código vulnerável 5	10

EMANIP

SUMÁRIO

BUFFER OVERFLOW EM WINDOWS	5
1 INTRODUÇÃO	5
2 SECURITY INTUITION	5
2.1 Metodologia científica aplicada para segurança	5
3 FUNÇÕES VULNERÁVEIS NA LINGUAGEM C	7
4 <i>BUFFER OVERFLOW</i>	9
4.1 Fuzzing.....	19
5 CONFIGURANDO O AMBIENTE VULNERÁVEL	19
5.1 Vulnserver	22
CONCLUSÃO	23
REFERÊNCIA	24

BUFFER OVERFLOW EM WINDOWS

1 INTRODUÇÃO

No campo de cibersegurança, compreender as vulnerabilidades que podem ser exploradas pelos adversários é fundamental para proteger sistemas e dados sensíveis. Um dos conceitos críticos que um profissional de segurança deve dominar é o *Buffer Overflow*, uma das mais simples técnicas de exploração que pode ter consequências devastadoras se não for adequadamente mitigada.

Neste capítulo, mergulharemos profundamente no conceito de *Buffer Overflow*, explorando o que é, como ocorre e como prevenir. Mais ao final veremos um breve *overview* sobre outras vulnerabilidades.

2 SECURITY INTUITION

Antes de começarmos a falar de fato sobre exploração, vamos aprender a criar uma mentalidade de pesquisador. Vai por mim, essa é a parte mais importante desse capítulo todo e é algo que você desenvolve ao longo do tempo. Vamos começar definindo a intuição.

O que é a intuição? A intuição é quando se compreende algo em um determinado momento, sem necessariamente realizar raciocínios complexos.

Como um pesquisador de vulnerabilidades, qualquer mensagem de erro pode gerar um indício de que algo não está certo, necessitando uma investigação mais aprofundada.

Isso ajuda o pesquisador a guiar de forma lógica sua pesquisa, priorizando certos componentes do sistema durante sua análise. E, é aí que a metodologia científica entra em jogo.

2.1 Metodologia científica aplicada para segurança

O que é a metodologia científica? Vamos relembrar e definir esse conceito.

A definição mais moderna sobre o método científico pode ser rastreada até o filósofo e psicólogo John Dewey, *How We Think* (ou *Como nós pensamos*, em português), onde ele descreve um sistema de investigação dividido em cinco etapas:

1. Uma dificuldade sentida.
2. Localizando e definindo o problema.
3. A sugestão de uma solução possível.
4. Refinando a sugestão.
5. Testando se a sugestão resolve o problema.

Essa metodologia se parece muito com o processo de desenvolvimento de software. Onde encontramos uma dificuldade em, por exemplo, limpar nossa caixa de e-mail. Localizamos e definimos que o problema está que recebemos muitas mensagens de spam. Criamos uma sugestão para solucionar o problema, como: “Eu poderia criar um script para executar todo dia 30 do mês para fazer a limpeza da minha caixa de e-mail?”. Refinamos a sugestão criando o script e posteriormente testando se isso ajuda a manter nossa caixa de e-mail limpa.

Já a metodologia científica é comumente ensinada como um processo contendo 6 etapas:

1. Fazer uma observação.
2. Fazer uma pergunta.
3. Formar uma hipótese.
4. Fazer uma previsão baseada na hipótese.
5. Testar a previsão.
6. Usar os resultados para fazer uma nova hipótese ou previsão.

Você provavelmente deve estar pensando como isso se aplica a segurança. Bom, normalmente, com o passar do tempo, isso acaba ficando algo bem natural. Quando apresentamos com um software, pensamos em um objetivo. Primeiro é feito uma observação de como o software se comporta em diferentes circunstâncias. Então, se é questionado quais vetores de ataques possivelmente podem causar alguma vulnerabilidade.

Assim, formulando algumas hipóteses sobre como podemos explorar uma determinada vulnerabilidade, conseguimos criar diferentes experimentos para testar

nossas hipóteses. Por fim, é coletado e analisado os dados para determinar a natureza e severidade da vulnerabilidade descoberta. Quando falarmos sobre o Buffer Overflow, você verá um exemplo claro disso.

3 FUNÇÕES VULNERÁVEIS NA LINGUAGEM C

Quando falamos sobre vulnerabilidades de memória, funções que realizam alguma operação com um determinado dado passado pelo usuário, por exemplo, ler o nome e sobrenome, o programa precisará armazenar esses valores temporariamente em uma variável. Linguagens como C ou C++ que necessitam de um gerenciamento manual da memória podem ser de grande risco se não forem performadas corretamente.

Vamos ver alguns exemplos de funções vulneráveis na linguagem C e entender o porquê é vulnerável.

Exemplo 1:

O código a seguir pede para que um usuário escreva seu sobrenome e em seguida armazena esse valor em um array de caracteres *last_name*.

```
char last_name[20];  
printf("Enter your last name: ");  
scanf("%s", last_name);
```

Código-fonte 1 – Exemplo de código vulnerável 1
Fonte: Elaborado pelo autor (2023)

O problema com esse código acima é que ele não está restringindo ou limitando o tamanho do sobrenome passado pelo usuário. Nesse exemplo, vemos que nosso buffer suporta 20 caracteres, caso o sobrenome de um usuário for maior que 20, digamos 25, então o buffer overflow irá acontecer.

Exemplo 2:

O código a seguir tenta criar uma cópia local de um buffer para executar algumas manipulações com os dados.

```
void manipulate_string(char* string) {  
    char buf[24];  
    strcpy(buf, string);  
    ...  
}
```

Código-fonte 2 – Exemplo de código vulnerável 2
Fonte: Elaborado pelo autor (2023)

No entanto, o programador não garante que o tamanho dos dados apontado por *string* se encaixe no *buf* local e copie os dados com a função **strcpy()** potencialmente perigosa. Isso pode resultar em uma condição de excesso de buffer se um invasor puder influenciar o conteúdo do parâmetro String.

Exemplo 3:

O código abaixo chama a função **gets()** a ser lida nos dados da linha de comando.

```
char buf[24];  
printf("Please enter your name and press <Enter>\n");  
gets(buf);
```

Código-fonte 3 – Exemplo de código vulnerável 3
Fonte: Elaborado pelo autor (2023)

No entanto, a função **gets()** é insegura, porque copia todas as entradas do *stdin* para o buffer sem verificar o tamanho. Isso permite que o usuário forneça uma sequência maior que o tamanho do buffer, resultando em uma condição de estouro.

Exemplo 4:

No exemplo a seguir, um servidor aceita conexões de um cliente e processa a solicitação do cliente. Depois de aceitar uma conexão com o cliente, o programa obterá informações do cliente usando o método **gethostbyaddr()**, copiará o nome do host do cliente que se conectou a uma variável local e produzirá o nome do host do cliente para um arquivo de log.


```
struct hostent* clienthp;
char hostname[MAX_LEN];

// create server socket, bind to server address and listen on socket
...

// accept client connections and process requests
int count = 0;
for (count = 0; count < MAX_CONNECTIONS; count++) {

    int clientlen = sizeof(struct sockaddr_in);
    int clientsocket = accept(serversocket, (struct sockaddr*)&clientaddr,
&clientlen);

    if (clientsocket >= 0) {
        clienthp = gethostbyaddr((char*)&clientaddr.sin_addr.s_addr,
sizeof(clientaddr.sin_addr.s_addr), AF_INET);
        strcpy(hostname, clienthp->h_name);
        logOutput("Accepted client connection from host ", hostname);

        // process client request
        ...
        close(clientsocket);
    }
}
close(serversocket);
```

Código-fonte 4 – Exemplo de código vulnerável 4
Fonte: Elaborado pelo autor (2023)

No entanto, o nome do host do cliente que conectou pode ser maior que o tamanho alocado para a variável local do nome do host. Isso resultará em um excesso de buffer ao copiar o nome do host do cliente para a variável local usando o método **strcpy()**.

4 BUFFER OVERFLOW

Agora vamos entender mais detalhadamente sobre essa vulnerabilidade, aplicando a metodologia que aprendemos acima. Você verá que o buffer overflow, uma vez compreendido, será a base para outras vulnerabilidades e exploração de memória.

```
#include <stdio.h>
#include <string.h>

int main(int argc, char* argv[])
{
    char buffer[64];

    if (argc < 2)
    {
        printf("Error: %s <args>\n", argv[0]);
        return 1;
    }

    strcpy(buffer, argv[1]);
    printf("Buffer: %s", argv[1]);

    return 0;
}
```

Código-fonte 5 – Exemplo de código vulnerável 5
Fonte: Elaborado pelo autor (2023)

Vamos usar o código acima para nos ajudar a criar uma boa visualização sobre o BOF.

Nesse caso, a função *main* primeiro define uma matriz de caracteres denominada *buffer* que pode conter até 64 caracteres. Como essa variável é definida dentro de uma função, o compilador C a tratará como uma variável local e reservará um espaço (64 bytes) para ela na pilha. Especificamente, esse espaço de memória será reservado dentro do quadro de pilha da função principal durante sua execução quando o programa for executado.

O programa, então, copia (*strcpy*) o conteúdo do argumento de linha de comando fornecido (*argv[1]*) para a matriz de caracteres do buffer. Observe que a linguagem C não termina com um caractere nulo (*\0*) ou, em outras palavras, uma matriz unidimensional de caracteres.

Por fim, o programa encerra sua execução e imprime na tela o conteúdo passado pelo usuário.

Quando chamamos esse programa, passamos argumentos de linha de comando para ele. A função *main* processa esses argumentos com a ajuda dos parâmetros, *argc* e *argv*, que representam o número de argumentos passados para o programa (passados como um inteiro) e um array de ponteiros para os próprios argumentos "string", respectivamente.

Se o argumento passado para a função principal tiver 64 caracteres ou menos, este programa funcionará conforme o esperado e sairá normalmente. Caso contrário, imprimirá o uso do programa.

strcpy antes	32 'A's copiados	80 'A's copiados
Endereço de destino da função strcpy()	Endereço de destino da função strcpy()	Endereço de destino da função strcpy()
Endereço de origem da função strcpy()	Endereço de origem da função strcpy()	Endereço de origem da função strcpy()
Buffer reservado	AAAAAAAAAAAAAAAAAAAA	AAAAAAAAAAAAAAAAAAAA
Buffer reservado	AAAAAAAAAAAAAAAAAAAA	AAAAAAAAAAAAAAAAAAAA
Buffer reservado	Buffer reservado	AAAAAAAAAAAAAAAAAAAA
Buffer reservado	Buffer reservado	AAAAAAAAAAAAAAAAAAAA
Endereço de retorno da função main()	Endereço de retorno da função main()	AAAA
Parâmetro da função main() 1	Parâmetro da função main() 1	AAAA
Parâmetro da função main() 2	Parâmetro da função main() 2	AAAA

Figura 1 - Stack layout antes e depois
Fonte: Elaborada pelo autor (2015)

Os efeitos dessa corrupção de memória dependem de vários fatores, incluindo o tamanho do estouro e os dados incluídos nesse estouro. Para ver como isso funciona em nosso cenário, podemos aplicar um argumento superdimensionado ao nosso aplicativo e observar os efeitos.

Vamos começar a trabalhar nossa metodologia nos baseando nas seguintes perguntas:

1. O que acontece se eu enviar mais dados do que é permitido?
2. Eu consigo controlar o EIP (*Extended Instruction Pointer*)?
3. Quantos bytes são necessários para controlar o EIP?

Abrindo esse simples binário no IDA, vamos ver sua representação em Assembly.

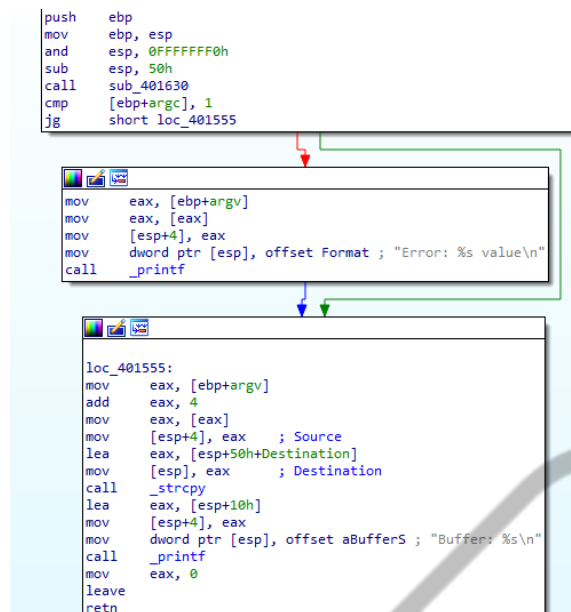


Figura 2 - Representação do código vulnerável em Assembly
Fonte: Elaborada pelo autor (2023)

Abrindo esse binário no x64dbg, veremos o que acontece quando passamos um valor menor que 64 e depois um valor maior que 64 bytes.

Para que você consiga executar um programa que necessite de argumento no x64dbg vá em *File > Change Command Line* e insira "AAAAAAAAAA" (10 'A's). Após isso execute o *programa uma vez*.

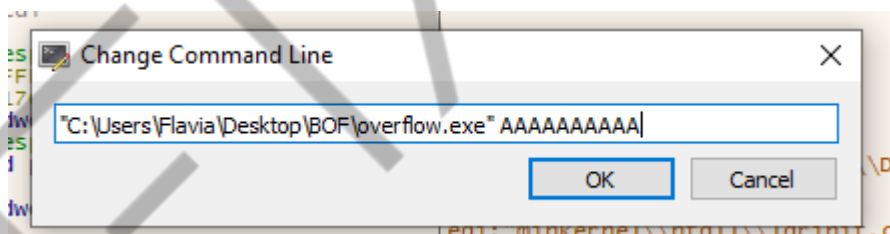


Figura 3 - Passando argumentos via debugger
Fonte: Elaborada pelo autor (2023)

🕸 overflow.exe - PID: 1540 - Module: overflow.exe - Thread: Main Thread 4824 - x32dbg

Figura 4 - Module: overflow.exe
Fonte: Elaborada pelo autor (2023)

Para encontrar nossa função main vamos buscar pela mensagem de erro. Essa é a forma mais simples de se encontrar a main nesse cenário. Para isso, use o seguinte atalho de teclado - *shift + D*. Com isso, seremos levados até uma aba onde estarão todas as strings encontradas no binário.

```

String A String
0040A000 "libgcc_s_dw2-1.dll"
0040A000 "libgcc_s_dw2-1.dll"
0040A013 "___register_frame_info"
0040A029 "___register_frame_info"
0040A044 "Error: %s value\n"
0040A055 "Buffer: %s\n"
0040A080 "Unknown error"
0040A19C "&Argument domain error (DOMAIN)"
0040A090 "matherr(): %s in %s(%g, %g), (retval=%g)\n"
0040A184 "Min-w64 runtime failure.\n"
0040A224 "VirtualProtect failed with code 0x%x"
0040A1F0 "VirtualQuery failed for %d bytes at address %p"
0040A1D0 "Address %p has no image-section"
0040A2AC "%d bit pseudo relocation at %p out of range, targeting %p, yielding the value %p.\n"
0040A280 "Unknown pseudo relocation bit size %d.\n"
0040A24C "Unknown pseudo relocation protocol version %d.\n"
00400080 "PE"
0040A300 "(null)"
0040A308 "L\"(null)\""
0040A316 "NaN"
0040A31A "Inf"
0040A31A "Inf"
0040A499 "NaN"
0040A490 "Infinity"
0040A660 "L\"msvcrt.dll\""
0040A676 "___lc_codepage_func"
0040A68A "___lc_codepage"
76746572 "L'd handle unless there is debugger"
000F0000 "A."

```

Figura 5 - Encontrando a mensagem de erro
Fonte: Elaborada pelo autor (2023)

Após encontrar a mensagem de erro, clique nela duas vezes e estaremos na main.

0040150C	FFD0	call eax	
0040150E	895C 08	mov dword ptr ss:[esp+8],ebx	
00401512	8B55 08	mov edx,dword ptr ss:[ebp+8]	
00401515	895424 04	mov dword ptr ss:[esp+4],edx	
00401519	890424	mov dword ptr ss:[esp],eax	
0040151C	E8 1F100000	call overflow.402540	[esp]:RtlIpv6AddressToStringA+1C6
00401521	8945 F4	mov dword ptr ss:[ebp-C],eax	
00401524	8B45 F4	mov eax,dword ptr ss:[ebp-C]	
00401527	8B5D FC	mov ebx,dword ptr ss:[ebp-4]	
0040152A	C9	leave	
0040152B	C3	ret	
0040152C	55	push ebp	
0040152D	8955	mov ebp,esp	
0040152F	83E4 F0	and esp,FFFFFFF0	
00401532	83EC 50	sub esp,50	
00401535	E8 F6000000	call overflow.401630	
0040153A	837D 08 01	cmp dword ptr ss:[ebp+8],1	
0040153E	7F 15	jg overflow.401555	
00401540	8B45 0C	mov eax,dword ptr ss:[ebp-C]	
00401543	8B00	mov eax,dword ptr ds:[eax]	
00401545	894424 04	mov dword ptr ss:[esp+4],eax	
00401549	C70424 55A04000	mov dword ptr ss:[esp],overflow.40A044	[esp]:RtlIpv6AddressToStringA+1C6, 40A044:"Error: %s value\n"
00401550	E8 98FFFFFF	call overflow.4014F0	
00401555	8B45 0C	mov eax,dword ptr ss:[ebp-C]	
00401558	8B00	add eax,4	
0040155D	894424 04	mov dword ptr ds:[eax]	
00401561	804424 10	lea eax,dword ptr ss:[esp+10]	
00401565	890424	mov dword ptr ss:[esp],eax	[esp]:RtlIpv6AddressToStringA+1C6
00401568	E8 C3690000	call <JMP.&mbcsncpy>	
0040156D	804424 10	lea eax,dword ptr ss:[esp+10]	
00401571	894424 04	mov dword ptr ss:[esp+4],eax	
00401575	C70424 55A04000	mov dword ptr ss:[esp],overflow.40A055	[esp]:RtlIpv6AddressToStringA+1C6, 40A055:"Buffer: %s\n"
0040157C	E8 6FFFFFFF	call overflow.4014F0	
00401581	B8 00000000	mov eax,0	
00401586	C9	leave	
00401587	C3	ret	
00401588	66:90	nop	
0040158A	66:90	nop	
0040158C	66:90	nop	
0040158E	66:90	nop	

Figura 6 - Função main encontrada no x64dbg
Fonte: Elaborada pelo autor (2023)

Adicione um breakpoint no *push ebp*. Só por garantia, caso seja necessário reiniciar a execução do programa.

Nosso interesse está na função **strcpy()** chamada itself, então podemos colocar um ponto de interrupção nela também. Para definir um ponto de interrupção na chamada da função *strcpy*, selecionamos a linha na janela de desmontagem no endereço 0x00401568 e pressionamos F2. Uma vez definido, o ponto de interrupção do endereço de memória ficará vermelho.

00401568	55	push ebp	
0040156D	8955	mov ebp,esp	
00401571	83E4 F0	and esp,FFFFFFF0	
00401575	83EC 50	sub esp,50	
0040157C	E8 F6000000	call overflow.401630	
00401581	837D 08 01	cmp dword ptr ss:[ebp+8],1	
00401586	7F 15	jg overflow.401555	
0040158A	8B45 0C	mov eax,dword ptr ss:[ebp-C]	[ebp-C]:BaseThreadInitThunk+19
0040158D	8B00	mov eax,dword ptr ds:[eax]	
00401590	894424 04	mov dword ptr ss:[esp+4],eax	
00401594	E8 98FFFFFF	mov dword ptr ss:[esp],overflow.40A044	40A044:"Error: %s value\n"
00401598	E8 98FFFFFF	call overflow.4014F0	[ebp-C]:BaseThreadInitThunk+19
0040159C	8B45 0C	mov eax,dword ptr ss:[ebp-C]	
0040159F	8B00	add eax,4	
004015A2	894424 04	mov dword ptr ds:[eax]	
004015A5	804424 10	lea eax,dword ptr ss:[esp+10]	
004015A8	890424	mov dword ptr ss:[esp],eax	
004015AB	E8 C3690000	call <JMP.&mbcsncpy>	
004015AE	804424 10	lea eax,dword ptr ss:[esp+10]	
004015B1	894424 04	mov dword ptr ss:[esp+4],eax	
004015B4	C70424 55A04000	mov dword ptr ss:[esp],overflow.40A055	40A055:"Buffer: %s\n"
004015B8	E8 6FFFFFFF	call overflow.4014F0	
004015BB	B8 00000000	mov eax,0	
004015BE	C9	leave	
004015C1	C3	ret	

Figura 7 - Adicionando um breakpoint na função strcpy
Fonte: Elaborada pelo autor (2023)

Em seguida, podemos continuar o fluxo de execução pressionando F9. Quase imediatamente, a execução para novamente pouco antes da chamada para a função strcpy, onde definimos nosso breakpoint (endereço: 0x00401568).

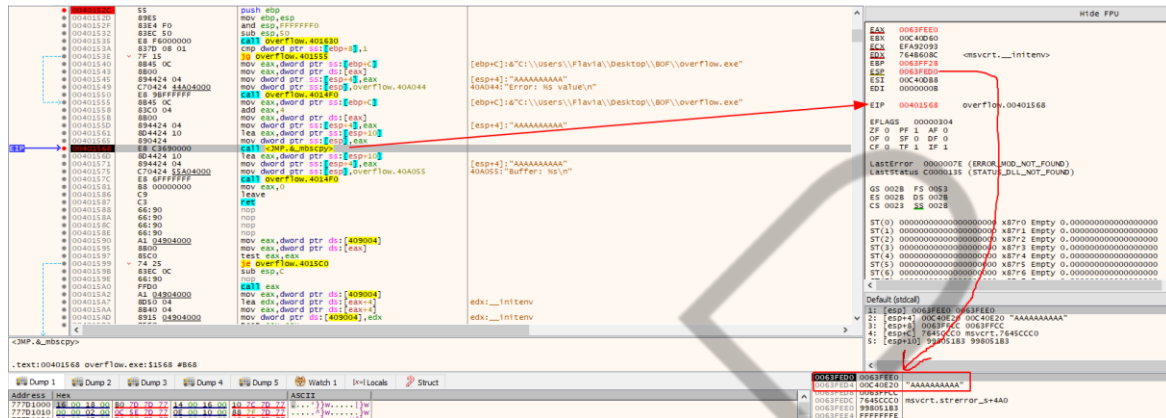


Figura 8 - Executando até a função strcpy
Fonte: Elaborada pelo autor (2023)

Conforme mostrado na figura “Executando até a função strcpy”, a execução foi pausada na função strcpy (endereço: 0x00401568). O EIP também é definido para este endereço, pois ele aponta para a próxima instrução a ser executada. Na janela da pilha, encontramos os dez caracteres "A" de nossa entrada de linha de comando e o endereço da variável de buffer de 64 bytes onde esses caracteres serão copiados (0x0063FEE0).

Podemos dar um novo passo para a chamada strcpy (com o atalho F7). Observe que os endereços na janela de instrução de montagem superior esquerda foram alterados porque somos novos dentro da função strcpy. Isso é observado pelo endereço destacado (0x758F6780) mostrado na figura “Stack layout antes da execução da função strcpy”.

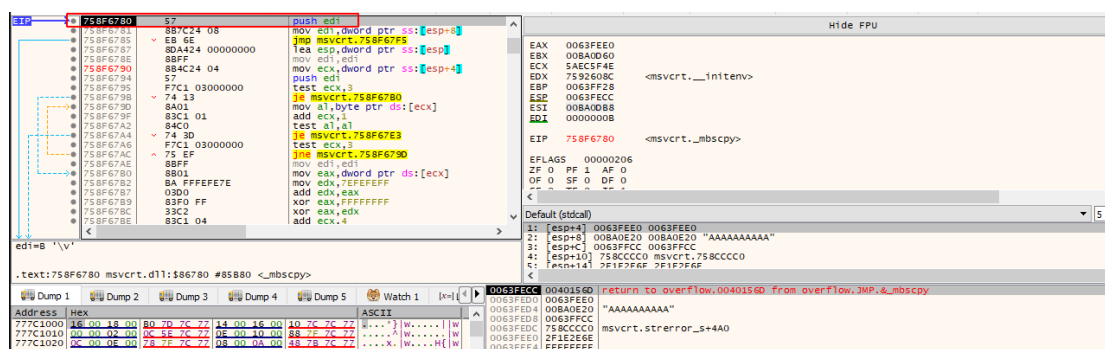


Figura 9 - Stack layout antes da execução da função strcpy

Fonte: Elaborada pelo autor (2023)

Agora, podemos clicar duas vezes no endereço de destino strcpy (0x0063FEE0) no painel de pilha para monitorar melhor as operações de gravação de memória que ocorrem nesse endereço, o que altera ligeiramente a exibição do painel da pilha.

Assim, vemos deslocamentos relativos (positivos e negativos) na coluna da esquerda ao invés de endereços reais:

\$-14	0040156D	return to overflow.0040156D from overflow.JMP.&mbstrcpy
\$-10	0063FEE0	
\$-C	008A0E20	"AAAAAAAAAA"
\$-8	0063FFCC	
\$-4	758CCCC0	msvcrt.strerror_s+4A0
\$ ==>	2F1E2E6E	
\$+4	FFFFFFFE	
\$+8	0063FF68	
\$+C	0040140F	return to overflow.0040140F from overflow.JMP.&onexit
\$+10	00401590	overflow.00401590
\$+14	77805DFE	return to ntdll.RtlAllocateHeap+3E from ntdll.RtlAllocateHeap+50
\$+18	00000000	
\$+1C	00000001	
\$+20	008A0DB8	
\$+24	00000008	
\$+28	0063FF68	
\$+2C	00401600	return to overflow.00401600 from overflow.00401400
\$+30	00401590	overflow.00401590
\$+34	00000000	
\$+38	00000008	
\$+3C	00000008	
\$+40	008A0DB8	
\$+44	00000008	
\$+48	0063FF68	

Figura 10 - Stack layout com offsets relativos antes da execução a função strcpy

Fonte: Elaborada pelo autor (2023)

Vamos dar uma olhada mais de perto nesta janela de pilha. Primeiro, observe que o deslocamento zero, agora realçado e anotado com o indicador de seta (==>), é o endereço 0x0063FEE0. Este é o início do buffer de destino onde a matriz de entrada de A's será copiada. Como diferenciamos um buffer de 64 bytes em nosso programa (buffer[64]), nosso buffer se estende do deslocamento 0 ao deslocamento +40, incluindo o terminador nulo para nossa matriz de caracteres.

Observe que há o que parecem ser endereços de retorno no buffer (offsets +C, +14 e +2C), bem como várias outras esquisitices. Como não inicializamos ou limpamos nosso buffer quando o definimos, esse espaço é preenchido com dados residuais que o depurador tenta interpretar. Quando chegar a hora de copiar nosso array de A's para o buffer, esses dados residuais serão substituídos.

Em seguida, observe o endereço de retorno 0x0040156D no topo da pilha. Este é o endereço para o qual retornaremos quando o strcpy for concluído e se correlacionar com a instrução **lea eax, dword ptr ss:[esp+10]**. Essa é a instrução

imediatamente após a instrução **call <JMP.&_mbscopy>** que nos trouxe aqui, e aquela instrução de chamada empurrou este endereço na pilha automaticamente para nós.

Neste ponto, podemos deixar a execução continuar até o final da função strcpy com o atalho **ctrl + F9**. Isso nos permitirá ver o resultado da função strcpy:

```

$-14 0040156D return to overflow.0040156D from overflow.JMP.&_mbscopy
$-10 0063FEE0 "AAAAAAAAAA"
$-C 008A0E20 "AAAAAAAAAA"
$-8 0063FFCC msvcrt.strerror_s+4A0
$-4 758CCCC0
$ ==> 41414141
$+4 41414141
$+8 00004141
$+C 0040140F return to overflow.0040140F from overflow.JMP.&_onexit
$+10 00401590 overflow.00401590
$+14 77805DFE return to ntdll.RtlAllocateHeap+3E from ntdll.RtlAllocateHeap+50
$+18 00000000
$+1C 00000001
$+20 008A0DB8
$+24 00000008
$+28 0063FF68
$+2C 00401600 return to overflow.00401600 from overflow.00401400
$+30 00401590 overflow.00401590
$+34 00000000
$+38 00000008
$+3C 00000008
$+40 008A0DB8
$+44 00000008
$+48 0063FF68
$+4C 004012F0 return to overflow.004012F0 from overflow.0040152C
  
```

Figura 11 - Stack layout com offsets relativos antes da execução a função strcpy (2)
Fonte: Elaborada pelo autor (2023)

Na figura acima, o strcpy copiou os dez caracteres “A” para a pilha (no buffer) e estamos claramente dentro do limite de buffer de 64 bytes imposto pelo tamanho da variável do buffer local declarado. Antes de prosseguir, anote mentalmente o endereço 0x004012F0 localizado no deslocamento 0x4C da variável do buffer. Este é o endereço de retorno da função principal, o endereço da instrução para a qual retornaremos assim que a função principal tiver concluído sua execução. Veremos essa ação em um momento.

Agora que a função strcpy concluiu sua execução e todos os dados foram copiados para o buffer de destino, é hora de retornar a execução para main por meio da instrução assembly RET.

```

758F6854 8B4424 08 mov eax,dword ptr ss:[esp+8]
758F6858 5F pop edi
758F6859 C3 ret
758F685A 66:8917 mov word ptr ds:[edi],dx
758F685D C647 02 00 mov byte ptr ds:[edi+2],0
758F6861 5F pop edi
758F6866 C3 ret
758F6867 66:8917 mov word ptr ds:[edi],dx
758F686A 8B4424 08 mov eax,dword ptr ss:[esp+8]
758F686E 5F pop edi
758F686F C3 ret
758F6870 8B17 mov byte ptr ds:[edi],dl
758F6872 8B4424 08 mov eax,dword ptr ss:[esp+8]
  
```

Registers: ESP: 0063FECC, ESI: 008A0DB8, EDI: 00000008, EIP: 758F6866, msvcrt.758F6866

Stack layout (bottom):

```

$-14 0040156D return to overflow.0040156D from overflow.JMP.&_mbscopy
$-10 0063FEE0 "AAAAAAAAAA"
$-C 008A0E20 "AAAAAAAAAA"
$-8 0063FFCC msvcrt.strerror_s+4A0
$-4 758CCCC0
$ ==> 41414141
$+4 41414141
$+8 00004141
  
```

Figura 12 - Retorno da função strcpy para main
Fonte: Elaborada pelo autor (2023)

Esta instrução RET "retira" o valor no topo da pilha (0x0040156D, deslocamento relativo -14) no registrador EIP, instruindo a CPU a executar o código naquele próximo local.

Se percorrermos esta instrução RET com a tecla F7, chegaremos de volta ao endereço da função principal 0x0040156D, como esperado, pois, este é o próximo endereço após a chamada strcpy.

A próxima instrução, **lea eax, dword ptr ss:[esp+10]** obterá o valor de retorno de strcpy e carregará no registrador EAX e, depois disso, o EAX será movido para [esp+4] como um parâmetro para a chamada printf. Após isso, eax recebe 0 que representa ou retorna 0 no código-fonte.

Neste ponto, chegamos ao epílogo da função principal. A função principal retornará a execução simples para variar o primeiro trecho de código criado pelo compilador (o ponto de entrada), inicialmente configurado e chamado de função principal em si.

A próxima instrução LEAVE coloca o endereço de retorno no deslocamento 0x4C (desde o início de nossa variável local de buffer) no topo da pilha. Em seguida, o assembly RET extrai o endereço de retorno da função principal do topo da pilha e executa o código ali.

Devemos demorar um pouco mais neste ponto para entender o quadro geral. Quando o strcpy copia a string de entrada (12 bytes) do argumento da linha de comando para a variável do buffer de pilha, ele começa a gravar no endereço 0x0063FF2C e em diante. Nesse caso, não há verificações gerais no tamanho da cópia em nosso código C, portanto, se passarmos uma string mais longa como entrada para nosso programa, dados suficientes serão gravados na pilha e, eventualmente, poderemos sobrescrever o endereço de retorno da função parent da rede localizada no deslocamento 0x4C da variável do buffer.

Isso significa que, se a substituição do endereço de retorno for realizada corretamente, o ponteiro da instrução ficará sob nosso controle, pois conterá alguns de nossos dados de argumento quando a função principal retornar ao pai.

Agora que entendemos como controlar o registrador EIP, vamos sobrescrevê-lo para com 80 bytes. O offset entre o endereço do buffer (0x0063FEE0) e o endereço

de retorno da função principal é de 76 (0x4C em hex) bytes, portanto, se fornecermos 80 'A's como argumento, todos eles devem ser copiados na pilha e gravados além dos limites do buffer atribuído no endereço de retorno de destino.

Para fazer isso, podemos reabrir o aplicativo com Arquivo > Abrir ou apenas pressionando F3 e inserir os 80 A's para os argumentos. Depois de colocar um ponto de interrupção na função strcpy e continuar a execução até o retorno, terminamos com um layout de pilha, como mostrado na figura abaixo.

```
0063FED0 0063FEE0 "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
0063FED4 00800E68 "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
0063FED8 0063FFCC 0063FFCC
0063FEDC 758CCCC0 msvcrt.strerror_s+4A0
0063FEE0 41414141
0063FEE4 41414141
0063FEE8 41414141
0063FEEC 41414141
0063FEF0 41414141
0063FEF4 41414141
0063FEF8 41414141
0063FEFC 41414141
0063FF00 41414141
0063FF04 41414141
0063FF08 41414141
0063FF0C 41414141
0063FF10 41414141
0063FF14 41414141
0063FF18 41414141
0063FF1C 41414141
0063FF20 41414141
0063FF24 41414141
0063FF28 41414141
0063FF2C 41414141
0063FF30 00000000
```

Figura 13 - O endereço de retorno da função antes da main (entripoint) está sobrescrito na *stack*
Fonte: Elaborada pelo autor (2023)

Se continuarmos a execução e prosseguirmos com a instrução RET na função principal, o endereço de retorno sobrescrito será exibido no EIP.

```
EAX 00000000
EBX 00B00D60
ECX FFFFFFFF
EDX 00000000
EBP 41414141
ESP 0063FF30
ESI 00B00E00
EDI 00000051 'Q'
EIP 41414141
```

Figura 14 - EIP overflow
Fonte: Elaborada pelo autor (2023)

Neste ponto, a CPU tenta ler a próxima instrução de 0x41414141. Como este não é um endereço válido no espaço de memória do processo, a CPU aciona um ACCESS VIOLATION, travando o aplicativo.

Mais uma vez, é importante ter em mente que o registrador EIP é usado pela CPU para direcionar a execução do código no nível do assembly. Portanto, obter um controle confiável do EIP nos permitiria executar qualquer código assembly que

desejarmos e, eventualmente, shellcode para obter um shell reverso no contexto do aplicativo vulnerável.

Como exercícios, tente repetir os passos até essa etapa.

1. Repita as etapas mostradas nesta seção para ver os 10 'A's copiados na pilha.
2. Forneça pelo menos 80 'A's e verifique se o EIP após o strcpy e printf conterá o valor 0x41414141.

4.1 Fuzzing

Existem três formas de se descobrir vulnerabilidades, sendo elas:

1. Tendo o código-fonte em mãos.
2. Realizando a engenharia reversa do código.
3. Utilizando ferramentas de Fuzzing.

O objetivo de Fuzzing é prover diferentes inputs que não serão devidamente tratados pela aplicação, com o intuito de quebrar a aplicação.

Algumas ferramentas para realizar o fuzzing são o Spiking e o AFL (American Fuzzy Loop). São ferramentas que nos ajudam a auditar vulnerabilidades em códigos.

5 CONFIGURANDO O AMBIENTE VULNERÁVEL

Vamos agora configurar nosso ambiente de exploração. Para isso precisaremos de uma máquina virtual com o Windows 10 para ser nosso alvo, e uma máquina Kali-Linux como nossa máquina de ataque.

Em nossa máquina Windows, precisaremos realizar algumas configurações. Utilizaremos o x64dbg para podermos depurar o programa que iremos explorar e utilizaremos um plugin chamado *mona*. Esse plugin é bastante utilizado para a exploração de binários.

Instale o IDAFree e o x64dbg, que são mais simples, e em seguida siga os passos para configurar o plugin corretamente no x64dbg.

Com o IDA e o x64dbg instalados em nossa máquina alvo, vamos agora configurar corretamente o plugin do mona, para nos ajudar durante nossa exploração.

Primeiro precisamos instalar o Python na sua versão 2.7, tanto na sua versão 32-bit quanto em 64-bit. Para isso vá em www.python.org/downloads/ e faça download da versão 2.7.0

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.2.0	Feb. 20, 2011	Download	Release Notes
Python 3.1.3	Nov. 27, 2010	Download	Release Notes
Python 2.7.1	Nov. 27, 2010	Download	Release Notes
Python 2.6.6	Aug. 24, 2010	Download	Release Notes
Python 2.7.0	July 3, 2010	Download	Release Notes
Python 3.1.2	March 20, 2010	Download	Release Notes
Python 2.6.5	March 18, 2010	Download	Release Notes
Python 2.5.5	Jan. 31, 2010	Download	Release Notes

[View older releases](#)

Figura 15 - Download da versão 2.7.0 do python
Fonte: Elaborada pelo autor (2023)

Download

This is a production release. Please [report any bugs](#) you encounter.

We currently support these formats for download:

- [Gzipped source tar ball \(2.7.0\) \(sig\)](#)
- [Bzipped source tar ball \(2.7.0\) \(sig\)](#)
- [Windows x86 MSI Installer \(2.7.0\) \(sig\)](#)
- [Windows X86-64 MSI Installer \(2.7.0\) \[1\] \(sig\)](#)
- [Mac Installer disk image \(2.7.0\) for OS X 10.5 and later \(sig\)](#). It contains code for PPC, i386, and x86-64.
- [32-bit Mac Installer disk image \(2.7.0\) for OS X 10.3 and later \(sig\)](#).
- [Windows help file \(sig\)](#)

Figura 16 - Download da versão 32 e 64 bits
Fonte: Elaborada pelo autor (2023)

Feito o download basta seguir a instalação padrão, next, next, finish. E, apenas na versão 64 que ele, o instalador, irá pedir para que crie um diretório, apenas altere para `C:\Python27-64\`.

Instalado corretamente nossa python, vamos agora configurar o plugin. Para podermos utilizar o script em python no x64dbg precisamos usar o x64dbgpy, esse programinha permite com que o debug interaja com o interpretador do python. Sua instalação é bem simples. Basta fazer download da sua última versão no github em releases. Caso o seu navegador realize o bloqueio do download, basta confirmar que você realmente quer fazer o download do arquivo e dar continuidade.

Feito o download do arquivo zip, extraia seu conteúdo e em seguida copie os devidos arquivos para o diretório da instalação do x64dbg. Nesse caso, só vamos trabalhar com a versão 32-bit, mas os mesmos passos se aplicam para versão de 64-bit.

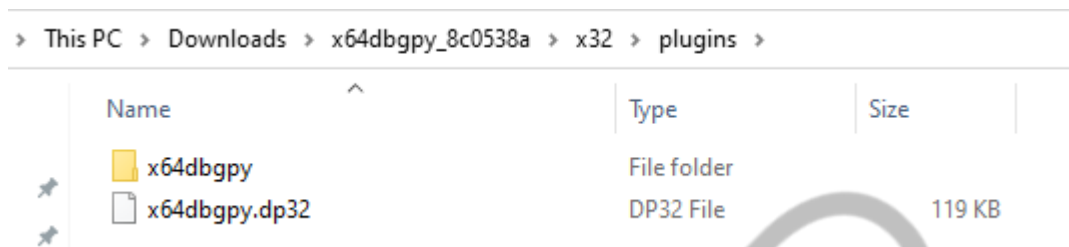


Figura 17 – Copiando os plugins
Fonte: Elaborada pelo autor (2023)

Em seguida, faça o download dos seguintes arquivos.

- [mona.py](#) – Salve em `x32 > plugins > x64dbgpy`.
- [pykd.py](#) & [x64dbgpylib.py](#) - Salve em `x32 > plugins > x64dbgpy`.
- E por fim, faça download do [clean_mona.py](#) – Salve em `x32 > plugins > x64dbgpy > x64dbgpy > autorun`.

Caso você tenha configurado corretamente aparecerá a opção de executar comando em python.

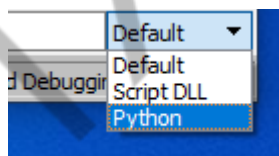


Figura 18 – Configurando o x64dbgpy
Fonte: Elaborada pelo autor (2023)

```

Initializing wait objects...
Initializing debugger...
Initializing debugger functions...
Setting JSON memory management functions...
Initializing Zydis...
Getting directory information...
Start file read thread...
Symbol Path: C:\Users\... \Downloads\x64dbg\release\x32\symbols
Allocating message stack...
Initializing global script variables...
Registering debugger commands...
Retrieving syscall indices...
Registering GUI command handler...
Registering expression functions...
Registering format functions...
Registering Script DLL command handler...
Starting command loop...
Initialization successful!
Loading plugins...
[PYTHON] Found valid PythonHome in the plugin settings!
[PYTHON] Python DLL: C:\WINDOWS\system32\python27.dll
[PYTHON] PythonHome: "C:\Python27"
[PLUGIN, x64dbgpy] Command "Python" registered!
[PLUGIN, x64dbgpy] Command "Pip" registered!
[PLUGIN, x64dbgpy] Command "PyRunScript" registered!
[PLUGIN, x64dbgpy] Command "PyRunScriptAsync" registered!
[PLUGIN, x64dbgpy] Command "PyRunGuiScript" registered!
[PLUGIN, x64dbgpy] Command "PyDebug" registered!
Syscall indices loaded!
Error codes database loaded!
Exception codes database loaded!
C:\Users\... \DOWNLO~1\x64dbg\release\x32\plugins\x64dbgpy
NTSTATUS codes database loaded!
Windows constant database loaded!
Reading notes file...
File read thread finished!
[PYTHON] stdout, stderr, raw_input hooked!
[PLUGIN] x64dbgpy v1 Loaded!
Handling command line...
"C:\Users\... \Downloads\x64dbg\release\x32\x32dbg.exe"

```

Figura 19 – x64dbgpy configurado corretamente

Fonte: Elaborada pelo autor (2023)

Em seguida, dê os seguintes comandos: *import mona*, e para testar se o mona está executando corretamente, insira um *mona.mona("help")*. Se tudo deu certo você terá a seguinte tela.

```

[+] Command used:
!mona help
'mona' - Exploit Development Swiss Army Knife - x64dbg (32bit)
Plugin version : 2.0.577
Written by Corelan - https://www.corelan.be
Project page : https://github.com/corelan/mona

<b> |-----|</b>
<b> |</b>
<b> |</b>
<b> | https://www.corelan.be |</b>
<b> | https://www.corelan-training.com |</b>
<b> | #corelan (Freengde IRC) |</b>
<b> |-----|</b>
<b></b>
Global options :
-----
You can use one or more of the following global options on any command that will perform
a search in one or more modules, returning a list of pointers :
-n : Skip modules that start with a null byte. If this is too broad, use
    option -cp nonull instead
-o : Ignore OS modules
-p <nr> : Stop search after <nr> pointers.
-m <module,module,...> : only query the given modules. Be sure what you are doing !
    You can specify multiple modules (comma separated)
    Tip : you can use -m * to include all modules. All other module criteria will be ignored
    Other wildcards : 'blah.dll' = ends with blah.dll, 'blah' = starts with blah,
    'blah' or 'blah' = contains blah
-cm <crit,crit,...> : Apply some additional criteria to the modules to query.
    You can use one or more of the following criteria :
    aslr,safeseh,rebase,nx,os
    You can enable or disable a certain criterium by setting it to true or false
    Example : -cm aslr=true,safeseh=false
    Suppose you want to search for p/p/r in aslr enabled modules, you could call
    !mona seh -cm aslr
-cm <crit,crit,...> : Apply some criteria to the pointers to return

```

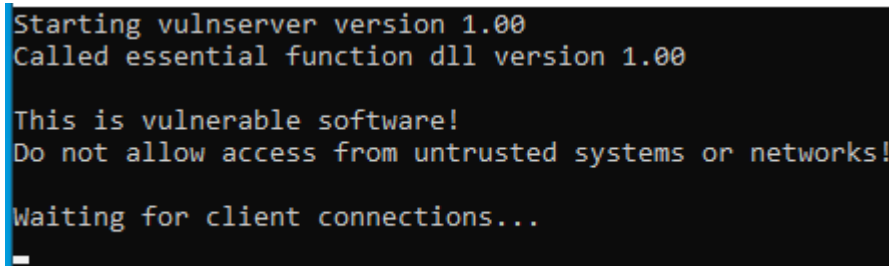
Figura 20 – Mensagem de help do plugin mona

Fonte: Elaborada pelo autor (2023)

5.1 Vulnserver

Para estudarmos sobre o buffer overflow, iremos utilizar um programa conhecido como [vulnserver](#), desenvolvido intencionalmente para a prática de BOF.

Basta fazer download do repositório e executar o binário vulnserver.exe. Ele abrirá um socket e ficará escutando na porta 9999.



```
Starting vulnserver version 1.00
Called essential function dll version 1.00

This is vulnerable software!
Do not allow access from untrusted systems or networks!

Waiting for client connections...
_
```

Figura 21 – Vulnserver em execução
Fonte: Elaborada pelo autor (2023)

CONCLUSÃO

Nesse capítulo conseguimos perceber que, além das ferramentas, ter uma mentalidade de pesquisador é essencial para embarcar nesse mundo de exploração de binários.

Entendemos algumas funções vulneráveis na linguagem C, e como isso pode causar problemas com BOF. Aprendemos também que, para explorar corretamente o BOF precisamos tomar controle sobre o registrador EIP.

REFERÊNCIA

DEWEY, John. **How We Think**. D.C. Heath & Company: Michigan, 2007.

EMASP